AIMERZ.ai
Aim.Act.Achieve

NOTES

Fundamentals of Java Programming -2

(Syntax, Control Structures, and Operators)

Naming Conventions, Reserved Keywords and Identifiers

Naming Conventions in Java

Naming conventions are the rules for how things like variables, methods, and classes should be named in Java. By following these rules, code becomes easier to read and understand.

Why Are Naming Conventions Important?

- **Easy to Read**: Clear names make it easier for others to understand code.
- **Avoids Mistakes**: Consistent naming helps prevent confusion and errors in large programs.
- **Professional**: Using standard names makes code look more organized and professional.

Common Naming Conventions in Java

1. **Class Names**:
    - Use *PascalCase* (start each word with a capital letter).
    - **Example**: Person, StudentDetails, CarModel
    - **Simple Example**: If creating a class for a **car**, it would be named CarModel.
2. **Method Names**:
    - Use *camelCase* (first word lowercase, and other words start with a capital letter).
    - **Example**: getTotalPrice(), startEngine(), sendEmail()
    - **Simple Example**: To create a method that **starts a car's engine**, it would be named startEngine().
3. **Variable Names**:
    - Use *camelCase* and make the names meaningful.
    - **Example**: age, totalAmount, maxSpeed
    - **Simple Example**: To track a **person's age**, the variable could be named age.
4. **Constant Names**:
    - Use all **UPPERCASE** letters with words separated by underscores.
    - **Example**: MAX_HEIGHT, PI, MAX_SPEED

- **Simple Example**: For setting a **maximum speed limit** in a car, it could be named MAX_SPEED.

Simple Rules for Naming in Java:

- **Be clear and meaningful**: The name should explain what the variable or method does.
- **Avoid using single letters** like x or y, unless in a short loop.
- **Use full words**: For example, totalPrice is better than tp.

Reserved Keywords:

Reserved Keywords in Java

In Java, **reserved keywords** are special words that the language uses, so they cannot be used to name variables, methods, or classes. These keywords have a specific purpose in Java.

What Are Reserved Keywords?

These are words that have a fixed meaning in Java. They cannot be used for naming anything else because Java uses them to perform specific tasks in the program.

Common Reserved Keywords in Java:

- **class**: Used to define a class.
- **if and else**: Used to make decisions in the code.
- **public, private, protected**: These control who can access the code.
- **for, while**: Used for loops to repeat code.
- **try, catch**: Used for handling errors in the code.
- **return**: Used to return a value from a method.

Here are a few more common reserved keywords:

public	**void**	**if**	**for**	**byte**	**int**
static	**for**	**while**	**abstract**	**assert**	**boolean**
break	**byte**	**case**	**char**	**lass**	**const**
continu	**default**	**do**	**double**	**else**	**enum**
extends	**final**	**finally**	**implements**	**import**	**new**

Identifiers in Java

All Java variables must be identified with unique names.
These unique names are called identifiers.

Identifiers are a way to name different parts of a program, like classes, methods, variables, and constants. Each type of identifier has its own rules.

What Are Identifiers?

Identifiers are the names given to elements in a Java program like:

- **Classes**
- **Methods**
- **Variables**
- **Constants**

Rules for Identifiers

- Identifiers can only contain **letters, digits, underscores (_), and dollar signs ($)**.
- They **cannot** start with a digit.
- Identifiers **cannot** be Java reserved keywords like if, else, class, etc.
- Java is **case-sensitive**: age and Age are different identifiers.

Examples:

- **Class Name**: CarModel
- **Method Name**: startEngine()
- **Variable Name**: maxSpeed
- **Constant Name**: MAX_SPEED

Using identifiers correctly makes it easier to understand and organize the program.

Type Casting and Truncation

What is Type Casting?

Type casting is the process of converting a value from one data type to another. There are two types of casting:

1. **Widening Casting** or **Implicit type casting** (automatic): This is done automatically when converting a smaller type to a larger type.
2. **Narrowing Casting** or **Explicit type casting**(manual): This is done manually when converting a larger type to a smaller type.

Widening Casting (Automatic Conversion)

Widening happens **automatically** in Java because there is no data loss. Here's how Java converts data automatically:

- **byte → short → int → long → float → double**

This means that Java can convert a byte to int, or int to double, without needing you to do anything.

Example of Widening Casting:

```Java
int myInt = 10;
double myDouble = myInt;  // Automatic conversion from int to double
System.out.println(myDouble);  // Output: 10.0
```

In this example, an integer 10 is automatically changed into a double 10.0. Nothing is lost in the conversion.

Narrowing Casting (Manual Conversion)

Narrowing happens when you convert a **larger type** to a **smaller type**. This needs to be done **manually** in Java because there is a risk of losing data.

- **double → float → long → int → short → byte**

When narrowing, Java cannot do this automatically, so you must specify the conversion with **type casting**.

Example of Narrowing Casting:

```Java
double myDouble = 9.78;
int myInt = (int) myDouble;  // Manual conversion from double to int
System.out.println(myInt);   // Output: 9
```

In this example, the value 9.78 (double) is manually cast to 9 (int). The decimal part is **lost** because int only stores whole numbers.

Truncation in Java

Why we are using Truncation with Real-World Example

Imagine you're counting the **number of pizzas** sold in a restaurant. The total number is stored as a **double** (like 25.99 pizzas), but when reporting, you only want to show whole pizzas.

When converting 25.99 to 25, you're **truncating** the .99 part and keeping only the whole number, 25.

What is Truncation?

Truncation happens when you convert a **decimal number** into a **whole number**. In Java, when you convert from a **floating-point** type like double or float to an **integer** type like int, the decimal part is **cut off** or **lost**.

Example of Truncation:

```Java

double myDouble = 7.89;
int myInt = (int) myDouble;  // Truncation occurs here
System.out.println(myInt);   // Output: 7
```

In this example, the value 7.89 is truncated to 7 when it's converted to int. The .89 part is simply removed.

Operators in Java

In Java, **operators** are special symbols that tell the computer to perform certain actions, like **adding numbers**, **comparing values**, or **assigning values**. Operators are an essential part of writing programs because they help you work with variables and data.

Arithmetic Operators: Basic Operators. **+, -, *, /**

Arithmetic operators are used to perform basic mathematical calculations like addition, subtraction, multiplication, etc.

Types of Arithmetic Operators:

- **+ (Addition)**: Adds two numbers.
  - Example: 5 + 3 = 8
- **- (Subtraction)**: Subtracts one number from another.
  - Example: 10 - 4 = 6
- *** (Multiplication)**: Multiplies two numbers.
  - Example: 6 * 7 = 42
- **/ (Division)**: Divides one number by another.
  - Example: 20 / 4 = 5

Example of Arithmetic Operators

```java
int sum = 5 + 3;  // Adds 5 and 3 to get 8
int sub = 5 - 3;  // Substract 5 and 3 to get 2
int mul = 5 * 3;  // Multiplies 5 and 3 to get 15
int div = 5 / 3;  // Divide 5 and 3 to get 1
```

Modulus Operator: %

This operator returns the remainder of a division operation

Example

```
Java
int mod = 5 % 3 // Divide 5 and 3 to get remainder 2
```

Assignment Operators

Assignment operators are used to assign values to variables.

Types of Assignment Operators:

- **= (Assign)**: Assigns the value on the right to the variable on the left.

  Example: x = 5

- **+= (Add and Assign)**: Adds the value on the right to the variable on the left and then assigns the result.

  Example: x += 3 is the same as x = x + 3

- **-= (Subtract and Assign)**: Subtracts the value on the right and assigns the result.

  Example: x -= 2 is the same as x = x - 2

Example 1:

```
Java
int a = 100;
a = a + 40;
System.out.println(a); // Result will be 140
```

We can rewrite above statement as

```
Java
int a = 100;
a +=  40;                // Assignment Operator
System.out.println(a); // Result will be 140
```

Above both operations perform the same operation which increases the value of a

Example 2:

```Java
int a = 5;
a += 3;  // a becomes 8
a -= 2;  // a becomes 6
```

Relational Operators:

Relational operators are used to **compare** two values. They return a true or false value based on the comparison.

Types of Relational Operators:

- **== (Equal to)**: Checks if two values are equal.
  - Example: 5 == 5 is true
- **!= (Not equal to)**: Checks if two values are not equal.
  - Example: 5 != 3 is true
- **> (Greater than)**: Checks if the left value is greater than the right.
  - Example: 7 > 4 is true
- **< (Less than)**: Checks if the left value is less than the right.
  - Example: 3 < 8 is true
- **>= (Greater than or equal to)**: Checks if the left value is greater than or equal to the right.
  - Example: 5 >= 5 is true
- **<= (Less than or equal to)**: Checks if the left value is less than or equal to the right.
  - Example: 2 <= 4 is true

```Java
== (equal to)
!= (not equal to)
> (greater than)
< (less than)
>= (greater than or equal to)
<= (less than or equal to)
```

Example

```java
public class RelationalOperatorsExample {
    public static void main(String[] args) {
        int a = 10;
        int b = 20;

        // == (equal to)
        System.out.println("a == b: " + (a == b));  // false

        // != (not equal to)
        System.out.println("a != b: " + (a != b));  // true

        // > (greater than)
        System.out.println("a > b: " + (a > b));    // false

        // < (less than)
        System.out.println("a < b: " + (a < b));    // true

        // >= (greater than or equal to)
        System.out.println("a >= b: " + (a >= b));  // false

        // <= (less than or equal to)
        System.out.println("a <= b: " + (a <= b));  // true
    }
}
```

Bitwise Operators: These operators manipulate the bits within an integer

~ Bitwise **unary Not** or Bitwise Complement.

This operator inverts all the bits of the given value

Example:
0101
Becoms
1010

Example:

```java
public class BitwiseComplement {
    public static void main(String[] args) {
        int number = 5; // Binary: 0101
```

```
        int result = ~number; // Invertrd Binary: 1010

        System.out.println("Original number: " + number);
        System.out.println("Bitwise complement: " + result);
    }
}
```

Output: -6

Bitwise **AND**: **&**
This operator compares two numbers bits and generates 1 if both bits are equal, else 0 if both bits are not equal
Example:
Binary of Decimal 42 is 00101010
Binary of Decimal 15 is 00001111
Apter Applying & Operator

```
  00101010
& 00001111
  00001010 which is 10
```

Code example

```
Java
int i = 42;
int j = 15;
System.out.println(i&j); // Output is 10
```

Bitwise **OR**: **|**

This operators compares the bits of the both decimal values and generates 1 if any of the bit is 1

Example:
Binary of Decimal 42 is 00101010
Binary of Decimal 15 is 00001111
After Applying | Operator

```
  00101010
| 00001111
  00101111 which is 47
```

Bitwise **XOR**: **^**

Each bit of the result is calculated by comparing the corresponding bits of the two operands:

If the bits are different, the result bit is 1.
If the bits are the same, the result bit is 0.

Example:
Binary of Decimal 42 is 00101010
Binary of Decimal 15 is 00001111
After Applying **^** Operator

```
    00101010
^   00001111
    00100101 which is 37
```

Code example

```java
int a = 5; // Binary: 0101
int b = 3; // Binary: 0011
int result = a ^ b; // Binary: 0110 (Decimal: 6)
```

Left Shift Operator: **<<**

This operator shifts the bits to the left to a specific number of times

Example:

```java
byte i = 62;
int j = i << 1;
System.out.println("Value of I: "+i);
System.out.println("Value of J: "+j);
```

Output is
Value of I: 64
Value of J: 124

Right Shift Operator: **>>**

This operator shifts the bits to the left to a specific number of times

Example:

```
Java
byte i = 62;
int j = i >> 1;
System.out.println("Value of I: "+i);
System.out.println("Value of J: "+j);
```

Output is
Value of I: 64
Value of J: 31

Short Circuit Logical Operators:

&& (and),

|| (or),

! (not).

These Combine multiple conditions. These operators are secondary versions of Bitwise AND and OR, and they generates only boolean expressions TRUE or FALSE

- The **AND (&&) operator** checks **two conditions** (A and B) and returns `true` only if **both A and B** are `true`.
- If **A is `false`**, it **will not check B** because the whole expression will already be `false` (since both conditions need to be `true` for the result to be `true`).

Examples of && Operator:

```
Java
  int i = 42;
  int j = 15;
  boolean k = i < j && j < i; // A = i < j, B = j < i
  System.out.println(k);
   // It always takes left (A) boolean value
// and result is i < j = true or i > b = false
```

- The **OR (| |) operator** checks **two conditions** (A and B). It returns `true` if **either A or B** is `true`.
- If **A is `false`**, it **will check B**. If **B is `true`**, the result will be `true`. If **B is `false`**, the result will be `false`.

Examples of || Operator:

```java
int i = 42;
int j = 15;
boolean k = i > j || j > i;
System.out.println(k); // Result is true
```

Logical Not: In Java, the ! operator is called the **logical NOT** operator. It is used to reverse the value of a boolean expression. If the expression is true, applying ! will make it false, and if the expression is false, applying ! will make it true.

Example of ! Operator

```java
public class NotOperatorExample {
    public static void main(String[] args) {
        boolean isRaining = false;

        // Using ! operator to reverse the boolean value
        if (!isRaining) {
            System.out.println("It is not raining. You can go outside.");
        } else {
            System.out.println("It is raining. You should stay inside.");
        }
    }
}
```

Increment and Decrement Operators

These operators are used to **increase** or **decrease** a variable's value by 1.

Types of Increment and Decrement Operators:

- **++ (Increment)**: Increases the value by 1.
  - Example: x++ is the same as x = x + 1
- **-- (Decrement)**: Decreases the value by 1.
  - Example: x-- is the same as x = x - 1

Example of Increment operator::

```Java
int x = 5;
x++;  // X will be calculated as x=x+1. And x become 6
```

Example of Decrement operator:

```Java
int y = 10;
y--;  // y will be calculated as y=y-1. And y become 9
```

Pre and Post Increment

In java these operators can be used for incrementations of the values but they can do extra operations like

first use then increase (X++)

Example 1 of Post Increment

```Java
int i = 42;
int j = i++;
System.out.println(j); // Output will be 42 only
```

first increase then use (**++**X)

Example 1 of Pre Increment

```
Java
int i = 42;
int j = ++i;
System.out.println(j); // Output will be 43
```

Example 2 of Pre Increment

```
Java
int i = 3;
int a = i++; // a = 3, i = 4
int b = ++a; // b = 4, a = 4
```

Practice Question:

What will be the output of the following program?

```
Java
public class Aimerz
{
    public static void main(String[] args)
    {
        int i = 11;
        i = i++ + ++i;
        System.out.println(i);
    }
}
//Output: 24
```

What will be the output of the below program?

```
Java
public class Aimerz
{
    public static void main(String[] args)
    {
        int i=0;
        i = i++ - --i + ++i - i--;
```

```
        System.out.println(i);
    }
}
//Output: 0
```

What is wrong with the below program? Why is it showing compilation error?

```Java
public class IncrementDecrementQuiz
{
    public static void main(String[] args)
    {
        int i = 11;
        int j = --(i++);
    }
}
```

Answer: Because, ++ or `--` can't be nested [`--(i++)`].

The ternary Operator

This operator acts like a three way operator that can perform an if, then, else statement It checks a condition and assigns a value based on whether the condition is true or false.

Basic Syntax of Ternary operator

```Java
variable = (condition) ? valueIfTrue : valueIfFalse;
```

Example:

```Java
int i = 42;
int j = 15;
```

```
// if i is grater than j or j is grater than j
String res = i > j ? "I is Grater " + i : "J is Grater " + j;
System.out.println(res);
```

In above example we are checking if i > j then print I grater or if i < j then print J is greater

Real-World Example of Operators

Imagine you're at a **grocery store**:

- **Arithmetic Operators**: You can use **+** to add the prices of items in your cart, or `/` to split the total bill between two people.
- **Relational Operators**: If you buy 5 items and you have a discount for more than 3 items, you can check `items > 3` to see if the discount applies.
- **Logical Operators**: If you want the discount to apply only if you have both more than 3 items **and** a membership card, you can use `&&` to check both conditions.
- **Assignment Operators**: If the total bill starts at `0`, you can use = to assign the total cost, and **+=** to add each item to the total.
- **Ternary Operator**: You can decide whether to apply a discount using the ternary operator, like:

```
Java

discount = (totalBill > 100) ? 10 : 0;
```

If your total is greater than `100`, you get a discount of `10`, otherwise, it's `0`.

Conditional Statements in Java

Introduction: In Java, conditional statements help the program make choices. It's like giving the computer instructions: "If this happens, do this.

If not, do something else." This is similar to real-life situations: "If you're tired, go to sleep; otherwise, stay awake."

What are Conditional Statements?

Conditional statements let a Java program decide what to do based on whether something is true or false. The most common ones are **if**, **else**, and **else if**. These allow different parts of the code to run based on conditions. For example:

- **If** the condition is true, the program does one thing.
- **Else** (otherwise), it does something different.

These statements help make the program smarter by allowing it to change its actions based on the input or data it gets.

Types of Java Conditional Statements

There are five types of Java conditional statements. All of these are widely used according to their syntaxes and use cases. We will discuss them further in detail so let us note them down below:

1. Java If Statement
2. Java If-Else Statement
3. Java If-Else-If Ladder Statement
4. Java Nested If Statement
5. Java Switch Statement

If Statement

The if statement checks if a **condition** is true. If it's true, the code inside the if block runs. If it's false, the code is skipped.

If a condition is true, the program runs a specific statement. This is called a one-way selection statement. When a condition is checked, and if it is true, the program runs the related code. If the condition is not true, nothing happens.

Basic Syntax:

```Java
if (condition) {
    // code to execute if the condition is true
}
```

Example:

```Java
int age = 18;
if (age >= 18) {
    System.out.println("You are an adult.");
}
//Output: You are an adult.
```

In this example, the program checks if age is 18 or older. If true, it prints "You are an adult."

Real-World Example:

Imagine you're at a movie theatre:

- **Condition**: If you're 18 or older, you're allowed to watch an adult-rated movie.
- If you're 18 or older, the code will run and you get a ticket.

If-Else Statement

An if-else statement is a way to control which actions a program takes based on certain conditions.

The if-else statement adds an **alternative**. If the condition is true, the first block of code runs. If the condition is false, the second block (the else part) runs.

Basic Syntax:

```java
if (condition) {
    // code to execute if the condition is true
} else {
    // code to execute if the condition is false
}
```

Example:

```java
int age = 16;
if (age >= 18) {
    System.out.println("You are an adult.");
} else {
    System.out.println("You are a minor.");
}
//Output: You are a minor.
```

In this example, if age is 18 or older, it will print "You are an adult." Otherwise, it prints "You are a minor."

Real-World Example:

Imagine you're deciding whether to carry an umbrella:

- **Condition**: If it's raining, take an umbrella. If it’s not raining, leave the umbrella at home.
- The decision is based on whether it’s raining or not.

Else If Statement

The else if statement allows you to check multiple conditions. It’s like having more than one choice. If the first condition is false, it checks the second one, and so on.

In an if-else-if statement, the conditions are checked one by one from top to bottom. As soon as a condition is found to be true, the program runs the code for that condition and skips the rest. If all the conditions are false, the last else statement is run.

Basic Syntax:

```Java
if (condition1) {
    // code if condition-1 is true
} else if (condition2) {
    // code if condition-2 is true
} else {
    // code if none of the conditions are true
}
```

Example:

```Java
int marks = 75;
if (marks >= 90) {
    System.out.println("Grade: A");
} else if (marks >= 75) {
    System.out.println("Grade: B");
} else if (marks >= 50) {
    System.out.println("Grade: C");
} else {
    System.out.println("Grade: Fail");
}
// Output: Grade: B
```

In this example, based on the value of marks, the program assigns a grade. If marks is 90 or more, the grade is A. If it's 75 or more but less than 90, the grade is B, and so on.

Real-World Example:

Imagine you're deciding what to wear based on the weather:

- **Condition 1**: If it’s cold, wear a jacket.
- **Condition 2**: If it’s warm, wear light clothes.

- **Condition 3**: If it's raining, take a raincoat.
- The decision changes based on the weather.

Nested If Statement

A **nested if** statement is when you put one if statement **inside another**. This is used when you need to check more than one condition within a block of code.

Nested if statements work by checking the first condition first. If the first condition is true, it then checks the second condition. Even if the second condition is false, the program can still continue if the first condition is true.

Basic Syntax:

```java
if (condition1) {
    if (condition2) {
        // code if both conditions are true
    }
}
```

Example:

```java
int age = 20;
boolean hasLicense = true;

if (age >= 18) {
    if (hasLicense) {
        System.out.println("You can drive.");
    } else {
        System.out.println("You need a licence to drive.");
    }
}

// Output: You can drive.
```

In this example, the program first checks if the `age` is 18 or older. Then, it checks if the person has a licence. Only if both conditions are true will it print "You can drive."

Real-World Example:

Imagine you're going on a trip:

- **Condition 1**: If you have enough money for the trip.
- **Condition 2**: If you have the required visa.
- Only if both conditions are met, you can go on the trip.

Switch Statement:

The **switch** statement is a little different from **if-else**. It helps check one value against many possible options. Instead of using many **if-else** checks, we can use **switch** to make the code easier to read.

When to Use a Switch Statement:

We use a **switch** statement when we want to compare a variable (like a number or a word) to multiple fixed values. It's like testing one thing against different possibilities.

How It Works:

- The **switch** checks the value of a variable.
- It compares the value with different cases.
- When it finds a match, it runs the code for that case.
- The **break** statement stops the program from running other cases.

This is very similar to **if-else** statements, but **switch** is better when we have many values to check, making the code simpler and cleaner.

Basic Syntax:

```java
switch (expression) {
    case value1:
        // Code to be executed if expression matches value1
        break;  // Optional, but prevents fall-through to the next case
    case value2:
        // Code to be executed if expression matches value2
        break;
    // You can have as many case statements as you want
    default:
        // Code to be executed if none of the cases match the expression
}
```

Example Syntax:

```Java
int day = 3;
switch(day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Other day");
}
//Output: Other day
```

In the above code we can see the switch is assigned on the variable 'day' and we are checking the case statement. If our case matches then the following case will execute

Output for above code is 'Other day' because case **1** and case **2** not matched

Let's another example

```Java
int day = 3;
switch(day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    case 3: System.out.println("Wednesday"); break;
    default: System.out.println("Other day");
}
// Output: Wednesday
```

Output for above code is 'Wednesday' because case 3 matched

Real-World Example:

Imagine you're ordering food at a restaurant:

- **Switch**: If you select option 1, you get pizza. If you select option 2, you get pasta. If none of the options match, you get an error message.

Exceptions for Using Switch Statement

Although the switch statement is useful, it cannot be used with all data types. It only works with **specific types**, and if we use unsupported types, we will get an error. Here's a list of types you **cannot use** in a switch:

1. **Floating-point types** (like float and double) – These types are not allowed in switch statements.

   Example: You cannot switch on a float variable like float num = 1.5f;

2. **Objects other than Strings** – You cannot switch on objects of custom classes unless you are using **Enums** or **Strings**.

   Example: You cannot switch on a Person object unless it's an Enum or String.

3. **Long type** – The long type is also not allowed in switch statements.

   Example: You cannot use a switch statement with long number = 100000L;

Switch statements mainly work with:

- **int** types (like byte, short, char, int)
- **String**
- **Enum types**

If you try to use unsupported types, Java will throw an error, and the program won't run.

Nested Switch Statements in Java

A **nested switch statement** is when we place one **switch** inside another. This works just like **if-else** statements, where we can have an **if** inside another **if**.

Since each switch has its own block of code, there won't be any confusion or conflict between the cases of the outer and inner switch. This makes it easier to organize decisions when there are multiple options within options.

Basic Syntax of Nested Switch Statement:

```java
switch (expression1) {
    case value1:
        // Outer switch case
        switch (expression2) {
            case valueA:
                // Inner switch case
                break;
            case valueB:
                // Inner switch case
                break;
            default:
                // Inner switch default case
        }
        break;
    case value2:
        // Outer switch case
        break;
    default:
        // Outer switch default case
}
```

Real-World Example: Food Ordering

Imagine you are at a restaurant:

- You have three main options on the menu: **Juice**, **Lunch**, or **Soft Drinks**.
- If you choose **Juice**, the waiter asks what kind of juice you want: **Mango Juice**, **Orange Juice**, or **Milkshake**.
- If you select **Lunch** or **Soft Drinks**, the waiter gives you your choice right away.
- If you pick something not on the menu, like "Pizza," the waiter tells you "You have selected nothing."

Java Code Example:

```java
public class NestedSwitchStatement {
    public static void main(String[] args) {

        // Restaurant Menu
```

```
        System.out.println("Welcome to the Restaurant!");
        System.out.println("Select an Item:");
        System.out.println("1: Juice");
        System.out.println("2: Lunch");
        System.out.println("3: Soft Drinks");

        int item1 = 1; // Let's say the user selects 1 for Juice

        switch (item1) {
            case 1:
                System.out.println("You selected Juice.");
                System.out.println("Which Juice?");
                System.out.println("Mango, Orange, or Milkshake");

                String item2 = "Mango"; // Let's say the user selects Mango

                switch (item2) {
                    case "Mango":
                        System.out.println("You ordered Mango Juice.");
                        break;
                    case "Orange":
                        System.out.println("You ordered Orange Juice.");
                        break;
                    case "Milkshake":
                        System.out.println("You ordered a Milkshake.");
                        break;
                    default:
                        System.out.println("Invalid choice of Juice.");
                }
                break;

            case 2:
                System.out.println("You selected Lunch.");
                break;

            case 3:
                System.out.println("You selected Soft Drinks.");
                break;

            default:
                System.out.println("You have selected nothing from the
menu.");
        }
    }
}
```

Output:
Welcome to the Restaurant!
Select an Item:
1: Juice
2: Lunch
3: Soft Drinks
You selected Juice.
Which Juice?
Mango, Orange, or Milkshake
You ordered Mango Juice.

Enhanced Switch Statement (Java 12 and Later)

In Java 12, the `switch` statement was updated to make it easier to use. One of the main changes is that you can now use it as an expression, meaning it can return a value directly instead of just executing blocks of code. This makes your code simpler and cleaner.

```Java
int day = 3;
String dayType = switch (day) {
    case 1, 7 -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default -> throw new IllegalArgumentException("Invalid day: " + day);
};
System.out.println(dayType);
// Output: Weekday
```

Step-by-Step Explanation of above code example

1. **Define a Variable**:
    - int day = 3; Here, we create a variable named day and assign it the value 3.
2. **Switch Expression**:
    - String dayType = switch (day) { — We start a switch expression using the variable day. We want to find out what type of day it is (either "Weekend" or "Weekday").
3. **Case Statements**:
    - case 1, 7 -> "Weekend"; — If day is 1 (Sunday) or 7 (Saturday), the result will be "Weekend".

- case 2, 3, 4, 5, 6 -> "Weekday"; — If day is any number from 2 to 6 (Monday to Friday), the result will be "Weekday".

4. **Default Case**:
   - default -> throw new IllegalArgumentException("Invalid day: " + day); — If day is not a valid number (like 0 or 8), it throws an error indicating that the day is invalid.
5. **Print the Result**:
   - System.out.println(dayType); Finally, we print the type of day. Since day is 3, it matches the "Weekday" case, so the output will be: Weekday

Java If-Else vs. Java Switch Statements

If-Else Statement	Switch Statement
Good for checking many conditions that can't be shown with one value.	Good for checking one value against many possible matches.
Looks at a true or false question to decide what to do next.	Looks at a value to see which case it matches.
Can handle complicated questions with many true/false checks.	Can only check a few specific values.
Can use different types of comparisons (like >, <, !=).	Can only use the equal sign (==) to compare values.

Loops in Java

Loops are a fundamental concept in programming. They allow us to repeat a block of code multiple times without writing it over and over again. This is useful when we want to perform the same action many times, like calculating totals, processing items in a list, or executing a set of instructions repeatedly until a certain condition is met.

In software programming, we often need to do the same task many times. There are two ways to do this:

1. **Write the same code over and over**: This is not a good idea because it makes the code messy and goes against the rule of "Don't Repeat Yourself" (DRY).
2. **Use loops**: Loops let us repeat a task without writing the same code again. This is a better and more efficient way.

When to Use Loops

Loops are useful in many situations. Here are a couple of examples:

1. **Accessing Array Elements**: When we want to look at each item in an array, we can use a loop. This is much faster than trying to get each item one by one, especially if the array is very big. Using a loop lets us do it in just a few seconds.
2. **Running Code Until a Condition is True**: If we want to keep running some code until a certain condition is met, but we don't know when that will happen, we can use a while loop in Java. This way, the code will keep running until the condition becomes true.

Java has three types of loops:

1. **For loops**
2. **While loops**
3. **Do-while loops**

For loop

The **for loop** is used when we want to go through items in a specific order. We use it when we know exactly when to start and when to stop. The way to write a for loop is very easy.

It consists of three parts: the initialization, the condition, and the increment/decrement.

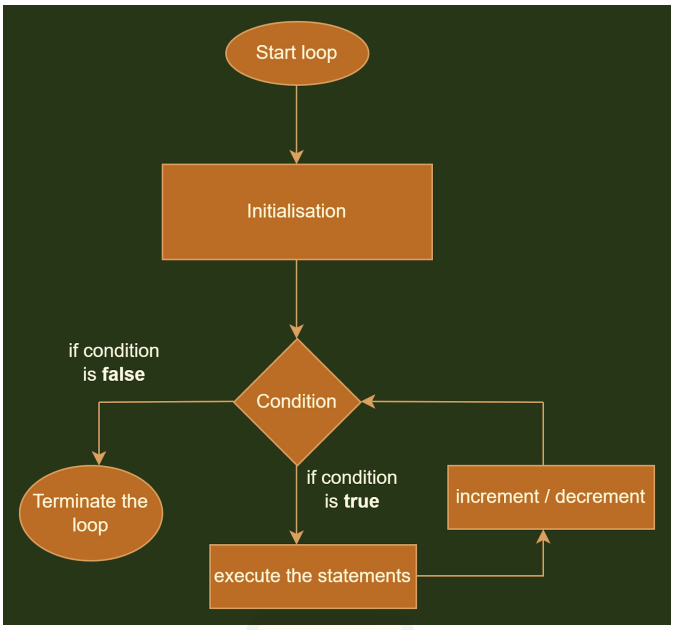
Basic Syntax

```java
for (initialization; condition; increment/decrement) {
    // code to be executed repeatedly
}
```

Explanation of the For Loop

1. **Initialization**: We start by setting a starting value for the variable that will be used in the loop. This step is important because the loop needs to know where to begin.
2. **Condition Checking**: Each time the loop runs, it checks if the variable meets the condition. If it does, the loop runs the code inside it. If not, the loop stops and exits immediately.
3. **Body of the Loop**: The body of the for loop is where we write the code that we want to run. The loop will only go into this part if the condition is true. We can also put another loop inside this body; these are called **nested loops**.
4. **Incrementing or Decrementing**: After running the code inside the loop, we change the value of the variable by either adding or subtracting from it. If we don't do this, the condition will always be true, and the loop will keep running forever. This is called an **infinite loop**.
5. **Loop Termination**: When the variable changes enough that it no longer meets the condition, the loop stops running and exits.

Control flow chart of for loop in Java:



Example of for loop : Printing first 5 digits starting from 0

```Java
for (int i = 0; i < 5; i++) {
    System.out.println("Count: " + i);
}
```

In the above example, the loop starts with i = 0, checks if i is less than 5, and prints the value of i. After each iteration, i is increased by 1. The output will be:

```Java
Count: 0
Count: 1
Count: 2
Count: 3
Count: 4
```

Example 2: Print the first 10 natural numbers.

step 1: declare num variable.
step 2: assign the initial value to 1.
step 3: set condition that num should not exceed 10
step 4: in the body of the loop print the ith natural number
step 4: increment the num value by 1 after every iteration.

```java
class ForLoopExample1 {
    public static void main(String[] args) {
       for(int iter_variable=1; iter_variable<=10; iter_variable++){
           System.out.println(iter_variable);
       }
    }
}
```

Lets see another example print a multiplication table for any given number

Step 1: declare a variable n and initialise it with 2
step 2: declare num variable in for loop.
step 3: and assign the initial value to 1 to num variable.
step 4: set condition that num should not exceed 10
step 5: in the body of the loop print the ith natural number multiplied by n
step 6: increment the num value by 1 after every iteration.

```java
      // Printing a Multiplication table for given number
      int n = 2;
      for (int i = 1; i <= 10; i++) {
          System.out.println(n + " X " + i + "= " + n * i);
      }
```

Output is:

```java
2 X 1= 2
2 X 2= 4
2 X 3= 6
2 X 4= 8
2 X 5= 10
2 X 6= 12
2 X 7= 14
2 X 8= 16
2 X 9= 18
2 X 10= 20
```

Real-World Example:

Think of a **countdown** timer. If you want to count down from 5 to 1, you can use a for loop:

- Start at 5.
- Continue until you reach 1.
- Decrease the count by 1 each time.

```Java
int n = 2;
for (int i = 5; i >=0; i--) {
      System.out.println(i);
}
```

Nested For Loop

A **nested for loop** means having one **for loop** inside another for loop. This is helpful when we need to repeat actions within a loop multiple times, especially when dealing with things like tables, grids, or working with 2D arrays.

How It Works:

- The **outer loop** runs first.
- For each time the outer loop runs once, the **inner loop** will run completely from start to finish.
- This repeats until the outer loop finishes all its rounds.

Example 1:

```Java
for (int i = 1; i <= 3; i++) {        // Outer loop
    for (int j = 1; j <= 2; j++) {    // Inner loop
        System.out.println(i + " " + j);
    }
}
```

Output:

1 1
1 2
2 1
2 2
3 1
3 2

Explanation:

1. The **outer loop** (i) runs 3 times (from 1 to 3).
2. Each time the outer loop runs, the **inner loop** (j) runs 2 times (from 1 to 2).
3. So, the total output will show pairs of numbers where i is from the outer loop and j is from the inner loop.

Real-Life Example:

Think of it like a clock:

- The **outer loop** is the **hour hand** (which moves from 1 to 12).
- The **inner loop** is the **minute hand** (which moves from 0 to 59 for each hour).

Example 2:

Lets see print multiplications table for any given number starting with table 1 and ending with table 5

Explanation to Write the Code:

1. **Set up the outer loop** (for (int i = 1; i <= 5; i++)):
   - This loop runs 5 times, representing numbers from 1 to 5 for which we want the multiplication table.
2. **Set up the inner loop** (for (int j = 1; j <= 10; j++)):
   - For each number in the outer loop, the inner loop runs 10 times to print the multiplication table from 1 to 10.
3. **Print the result** (System.out.println(i + " X " + j + " = " + i * j)):
   - This statement prints the multiplication in the format like 1 X 1 = 1, 1 X 2 = 2, and so on.
4. **Add space between tables** (System.out.println()):
   - This adds a blank line after each table for better readability.

This code will print the multiplication tables for numbers 1 to 5.

```java
// Printing a Multiplication table starting from 1 to 5
int n = 1;
for (int i = 1; i <= 5; i++) {
      for (int j = 1; j <= 10; j++) {
            System.out.println(i+" X " + j + " = " + i * j);
      }
      System.out.println();
}
```

Output will be

1 X 1 = 1 1 X 2 = 2 1 X 3 = 3 1 X 4 = 4 1 X 5 = 5 1 X 6 = 6 1 X 7 = 7 1 X 8 = 8 1 X 9 = 9 1 X 10 = 10	2 X 1 = 2 2 X 2 = 4 2 X 3 = 6 2 X 4 = 8 2 X 5 = 10 2 X 6 = 12 2 X 7 = 14 2 X 8 = 16 2 X 9 = 18 2 X 10 = 20	3 X 1 = 3 3 X 2 = 6 3 X 3 = 9 3 X 4 = 12 3 X 5 = 15 3 X 6 = 18 3 X 7 = 21 3 X 8 = 24 3 X 9 = 27 3 X 10 = 30	4 X 1 = 4 4 X 2 = 8 4 X 3 = 12 4 X 4 = 16 4 X 5 = 20 4 X 6 = 24 4 X 7 = 28 4 X 8 = 32 4 X 9 = 36 4 X 10 = 40	.5 X 1 = 5 5 X 2 = 10 5 X 3 = 15 5 X 4 = 20 5 X 5 = 25 5 X 6 = 30 5 X 7 = 35 5 X 8 = 40 5 X 9 = 45 5 X 10 = 50

While Loop

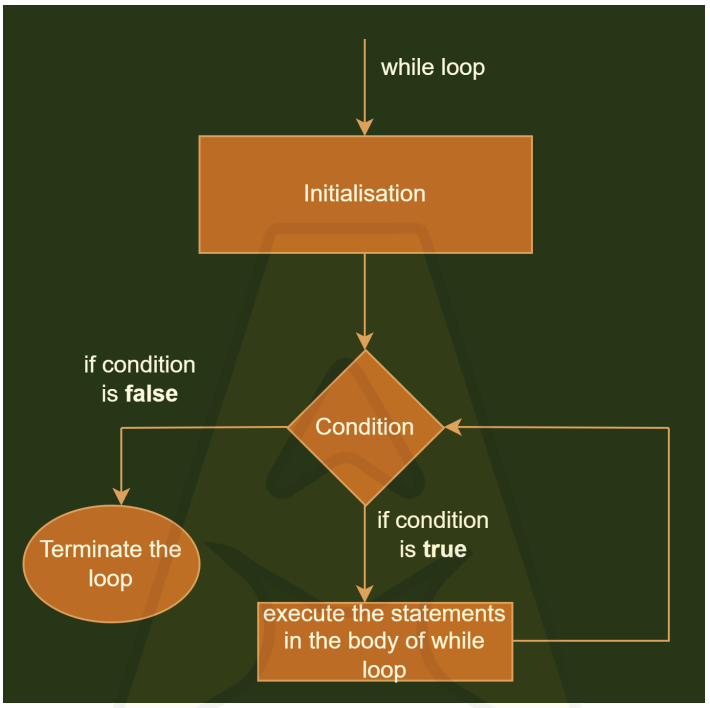
A **while loop** is used when we want to repeat a block of code as long as a certain condition is **true**. We may not know how many times we will need to loop when we start.

The **while loop** is a common loop in Java. We use it when we don't know exactly when the loop should stop. The while loop keeps running the code inside it as long as a certain condition is true. Once the condition becomes false, the loop stops, and we exit it.

Basic Syntax:

```java
while (condition) {
    // code statements
    // increment or decrement
}
```

Control flow of while loop



How It Works:

1. The while loop starts by checking the condition. If the condition is true, the loop runs the code inside it.
2. If the condition is false, the loop does not run at all and stops immediately. That's why the while loop is also called an **entry-control loop**.

Explanation:

- The loop checks the condition first. It only goes into the body of the loop if the condition is true. If the condition is false, the loop ends.
- You can use more than one condition in a while loop. Depending on the conditions, the loop will either run or stop.

- You don't have to use a variable that changes each time, but you need a way to stop the loop using a **break statement**. Otherwise, the while loop will keep running forever until the program stops.

Example 1: Print first five numbers starting with 0

```
Java
int count = 0;
while (count < 5) {
    System.out.println("Count: " + count);
    count++;
}
```

In this example, the loop continues to run as long as `count` is less than 5. The output will be the same as before:

```
Java
Count: 0
Count: 1
Count: 2
Count: 3
Count: 4
```

Real-World Example:

Imagine you are waiting for a bus. You keep checking if the bus has arrived:

- **Condition**: The bus has not arrived.
- Keep waiting (looping) until the bus shows up.

Example 2: print the first 10 natural numbers using a while loop.

```
Java
class whileLoopExample1 {
    public static void main(String[] args) {
        int i=1;  // initialise i=1
        // iterate until i is less than or equal to 10
        while(i<=10){
            System.out.println(i);
            i++; // increment i by 1
```

```
        }
    }
}
```

Infinite Loop Example in a While Loop

An **infinite loop** happens when a condition in a loop never becomes false, causing the loop to run forever. This can happen if the condition is always true or if there's no code inside the loop that can stop it.

Syntax:

```Java
while (true) {
    System.out.println("This will print forever!");
}
```

Explanation:

- **Condition is Always True**: In the example above, the condition is true, which means the loop will **never stop** running.
- **No Exit Condition**: Since there's no code to break the loop or change the condition, it will keep printing the message "This will print forever!" again and again.

How to Stop the Loop:

You can use a **break** statement inside the loop to stop it when needed, but without it, the loop will continue running infinitely.

Infinite Loop Example 2:

```Java
int i = 1;
while (i > 0) {
    System.out.println("Looping... i is " + i);
    i++; // i keeps increasing, so the condition i > 0 is always true
}
```

This type of loop can quickly consume memory or crash your program if it goes on too long, so always ensure you have a way to break out of it!

Do-While Loop

The **do-while loop** is a little different from the regular while loop. In a while loop, the condition is checked first, and the code runs only if the condition is true. In a do-while loop, the code inside the loop runs at least once before checking the condition. After the first run, it checks the condition just like a while loop.

Syntax:

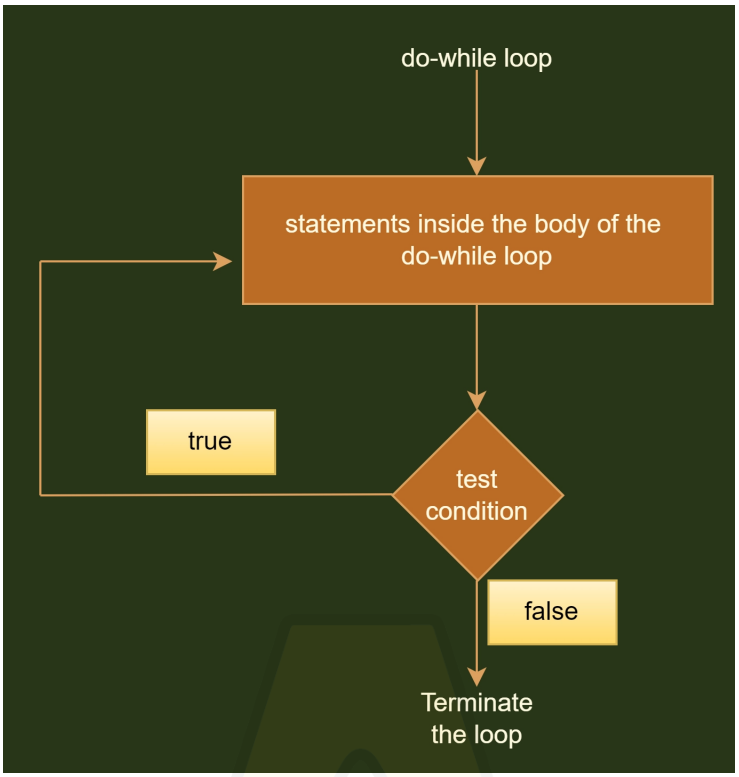
```Java
do {
    // code statements
    // increment or decrement
} while (condition);
```

How It Works:

1. The do-while loop will always run the code inside it at least once.
2. After running the code, it checks the condition. If the condition is true, the loop runs the code again.
3. If the condition is false, the loop stops and exits.

Since the do-while loop does not check the condition before running the code (unlike the while loop), it is called an **exit control loop**.

Control flow of do-while loop



Example of do-while loop

```
Java
int count = 0;
do {
    System.out.println("Count: " + count);
    count++;
} while (count < 5);
```

In this case, the output will also be:

```
Java
Count: 0
Count: 1
Count: 2
Count: 3
Count: 4
```

Let's take another example

```Java
int n = 1;

do {
      System.out.println(n);
      n++;
} while (n == 5);
```

Output is **1**

In above code we can see that even if the while condition is false do block has been executed for one time

Real-World Example:

Think about a **game** where you roll a die:

- You roll the die (do it once).
- Check if you rolled a 6 (condition).
- If not, roll again until you get a 6.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve